Assignment 2

ADVANCED SIMULATION

EPA133A

Authors:

Niels van den Dool (5026717)
Eli Kapteijn (6587380)
Nick Schreurs (6581951)
Lieke Wahlen (5179564)
Mirte Wildeboer (5326265)

March 6, 2026

TU Delft
Delft University of Technology

Contents

1 Data Preparation **2**
1.1 Datasets . . . 2
1.2 Challenges of the data . . . 2
1.3 Data Processing . . . 2
1.4 Data Selection . . . 2

2 Model Architecture **4**
2.1 Model improvements . . . 4
2.1.1 Model input: Bridge condition . . . 4
2.1.2 Bridge component . . . 4
2.1.3 Vehicle class . . . 5
2.1.4 Data collection: Average driving time . . . 6

3 Experiments & Scenarios **7**
3.1 Scenario description . . . 7

4 Results & Discussion **8**
4.1 Results of scenarios . . . 8
4.2 Differences and comparison between scenario outcomes . . . 8

5 Reflection **10**
5.1 Reliability of replications . . . 10
5.2 Limitations and possible improvements of the model . . . 10

References **11**

Acknowledgements **12**

1 Data Preparation

1.1 Datasets

Two datasets were provided. The first dataset, *_roads3.csv*, contains an overview of roads and their associated LRPs. The second dataset, *BMMS_overview.xlsx*, contains information about bridges, including their physical characteristics.

1.2 Challenges of the data

Several issues were identified in the datasets which required pre-processing before they could be used in the simulation model.

First, the roads dataset does not contain explicit information about road segment lengths. Instead, it provides LRPs that indicate locations along the road without specifying the distance between them. Furthermore, the dataset contains many different types of LRPs, not only bridges. Since the simulation model requires a simplified network consisting of sources, links, bridges, and sinks, these LRPs could not be used directly.

Additionally, inconsistencies were identified in the 'gap' column of the roads dataset. The values *BS* (Bridge Start) and *BE* (Bridge End) were occasionally missing or inconsistent with the bridge names. After visual inspection, the 'name' column was considered more reliable and was therefore used instead of the gap indicators.

Finally, the LRP names between the roads dataset and the bridge dataset did not match, making a direct merge between the datasets impossible.

1.3 Data Processing

Because of these challenges, processing of the N1 Road and bridges dataset was necessary, as Marsden and Pingry (2018) argue, even advanced models are not useful without high-quality input data.

So, to prepare the data for the simulation model, the datasets were transformed into a simplified network structure. The simulation requires four types of elements:

- Source: starting point of the road
- Bridge: bridge or culvert infrastructure
- Link: road segments between nodes
- Sink: end point of the road

The approach therefore consisted of identifying all bridges along the N1 road and treating the road segments between consecutive bridges as links.

1.4 Data Selection

The roads dataset was filtered to include only the N1 road. The minimum and maximum chainage values were used to determine the source and sink nodes.

Bridge names were standardized to simplify duplicate detection. Additionally, coordinates were rounded to ensure that bridges with nearly identical locations could be detected. Duplicate bridges were then removed based on chainage and rounded location.

Each bridge was then assigned the model type *bridge*. The source and sink nodes were manually added to the dataset. The source and sink nodes were then combined with the bridge dataset.

Many bridges appear twice in the dataset because they are recorded separately for the left and right side of the road. Since the simulation only drives one way, these bridges need to be removed. These duplicates were removed by comparing the prefix of the bridge names using a regex expression. An assumption here is that the bridges are approximately the same length, since the first one is always kept, instead of finding the right one. See figure 1

Links representing road segments were generated between each pair of consecutive nodes. The location of each link was approximated as the midpoint between the coordinates of the two adjacent nodes, as seen in figure 2. The length of the link was calculated as the difference in chainage between the nodes.

```python
final_df = (
    final_df
    .assign(
        prefix=(
            final_df["name"]
            .str.replace('.', '')
            .str.lower()
            .str.strip()
            .str.extract(r"^([^(]+)", expand=False)
        )
    )
    .drop_duplicates(subset="prefix", keep="first")
    .drop(columns="prefix")
)
```

Figure 1: Code excerpt of adjustments to DataFrame from Jupyer notebook 1_data_cleaning.ipynb

```python
for j in range(len(final_df)):

    row = final_df.iloc[j]

    prev = final_df.iloc[j-1]

    lat_link = (row['lat'] + prev['lat']) / 2
    lon_link = (row['lon'] + prev['lon']) / 2

    link_length = (row['chainage'] - prev['chainage']) * 1000
```

Figure 2: Code excerpt of how links are produced for the DataFrame from Jupyter notebook 1_data_cleaning.ipynb

Because the chainage values are expressed in kilometres, they were converted to meters for the simulation model. The resulting dataset was exported as a CSV file.

The final data file can be found in figure 3:

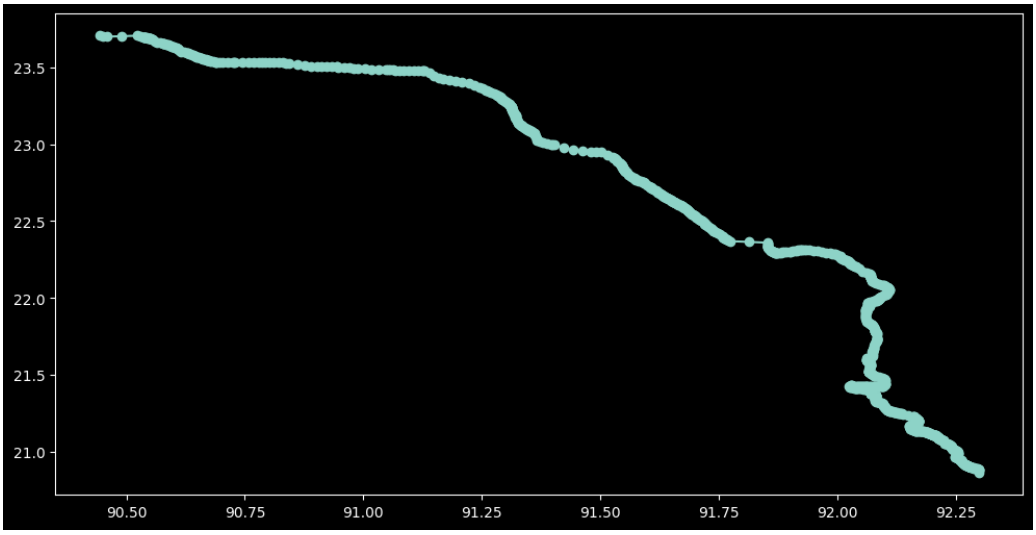

Figure 3: Visualization of the generated road network of bridges and links along the N1.

The resulting file follows the same structure as the demo datasets provided with the simulation model and can therefore be directly used as input for the simulation.

2 Model Architecture

The model uses a component-based decomposition (sources, links, bridges, sinks) to manage system complexity. Breaking down a complex system into manageable units is a central concept in modeling. As Hofmann (2004) describes, this approach offers several advantages, such as enabling parallel development and execution of the model, improving reusability, and limiting the extent of modifications required when changes are made.

2.1 Model improvements

Adjustments have been made to the `model.py` and the `components.py` to make sure bridges are able to break down given a chance of breaking (as provided in scenarios 1 to 8). In order to include these changes, the bridges must also include their category (in the code referred to as 'condition').

2.1.1 Model input: Bridge condition

As mentioned in Section 1, the condition of the bridges needs to be taken into account. After adjusting the input-file, this requires to change the model input as well.

The parameter 'scenario' was added to the initialization function of the `BangladeshModel`. The parameter was added with a base case of each condition having a chance of 0% of breaking down. By letting the user provide the custom configuration, there's a possibility to choose any scenario possible.

```
class BangladeshModel:
    def __init__(..., scenario='A': 0, 'B': 0, 'C': 0, 'D': 0):
    ...
    ...
    self.scenario = scenario

```

Figure 4: Code excerpt of model initialization with the addition of scenario as a parameter in blue.

Additionally the parameter is added to the Model initialization function by initializing the parameter. Both changes can be seen in figure 4

```
elif model_type == 'bridge':
    agent = Bridge(..., condition=row['condition'], broken_chance=self.scenario[row['condition']])

```

Figure 5: Code excerpt for generating bridges with the additions highlighted in blue.

2.1.2 Bridge component

In the `generate_model()` function in the `class BangladeshModel` the sources, sinks, links and bridges are generated as infrastructure. With the addition of the chance of a bridge being broken, the proper adjustments have to be made to the `class Bridge`.

When initializing the `Bridge` the component receives two extra parameters: the condition of the bridge (given by 'A', 'B', 'C' or 'D') and the chance of a bridge being broken in the given scenario.

```python
        def __init__(self, unique_id, model, length=0, name='Unknown', road_name='Unknown', condition='Unknown', broken_chance=0):
            super().__init__(unique_id, model, length, name, road_name)

            self.condition = condition
            self.broken_chance = broken_chance
            self.state = self.set_broken_bridge()
            self.delay_time = self.set_delay_time()

            class State(Enum):
                HEALED = 1
                BROKEN = 2

            def set_broken_bridge(self):
                if self.random.random() < self.broken_chance:
                    return Bridge.State.BROKEN
                else:
                    return Bridge.State.HEALED


```

Figure 6: Code excerpt with additions to the Bridge class initialization and the state

To be able to keep track of whether a bridge is broken, a state was added with Enum to set the state to either broken or whole (programmed as the arbitrarily chosen HEALED state). The broken state is determined by the probability (from the given probability for the condition in the given scenario) in the `set_broken_bridge()` function as shown in figure 5

```python
        def set_delay_time(self):
            if self.State == Bridge.State.HEALED:
                return 0
            else:
                match self.length:
                    case n if n < 10:
                        return self.random.uniform(10,20)
                    case n if n < 50:
                        return self.random.uniform(15,60)
                    case n if n < 200:
                        return self.random.uniform(45, 90)
                    case n if n >= 200:
                        return self.random.triangular(60,240,120)

```

Figure 7: Code excerpt of the set_delay_time() function inside the Bridge class.

2.1.3 Vehicle class

To determine the delay time caused by a defect bridge, the `get_delay_time()` function is called by a vehicle when their next infrastructure in the model is a bridge in the `drive_to_next()` function. This call to the function and the `get_delay_time()` are shown in figure 8

```
        if isinstance(next_infra, Sink):
            # arrive at the sink
            self.arrive_at_next(next_infra, 0)
            self.removed_at_step = self.model.schedule.steps

            driving_time = self.removed_at_step - self.generated_at_step
            self.model.completed_trip_times.append(driving_time)
            self.location.remove(self)

        elif isinstance(next_infra, Bridge):
            if next_infra.state == Bridge.State.BROKEN:
                self.waiting_time = next_infra.get_delay_time()
            if self.waiting_time > 0:
                # arrive at the bridge and wait
                self.arrive_at_next(next_infra, 0)
                self.state = Vehicle.State.WAIT
                return
            # else, continue driving

```

Figure 8: Code excerpt of the additions to the drive_to_next() function in the Vehicle class.

2.1.4 Data collection: Average driving time

In order to gather the information from the model, mesa's DataCollector class was used to report the average driving time throughout the model. The model reporter calculates the average driving time of all vehicles exiting the model through a sink at every time step. With this, the average driving time can be examined at every time step and only when a truck has finished the drive, one will see a change in the reported average driving time.

The data collector and the method it calls can be found in figures 9 and 10.

```
    self.datacollector = DataCollector(
            model_reporters="Average_Driving_Time":  compute_avg_driving_time
        )
```

Figure 9: Code excerpt of the compute_avg_driving_time() function in the BangladeshModel class.

```
        def compute_avg_driving_time(model):
        """Calculates the average driving time of completed trips."""
        if len(model.completed_trip_times) == 0:
            return 0.0
        return sum(model.completed_trip_times) / len(model.completed_trip_times)

```

Figure 10: Code excerpt of the compute_avg_driving_time() function in the BangladeshModel class.

3 Experiments & Scenarios

3.1 Scenario description

For the model, several scenarios are developed. The trucks are assumed to travel without traffic congestion or other obstacles, except for bridges along the route. These bridges may fail, which forces the truck to wait until the bridge has been repaired. The bridge condition data were used to determine the probability of bridge failure. The distribution of bridge conditions and their frequencies are shown in Figure 11.

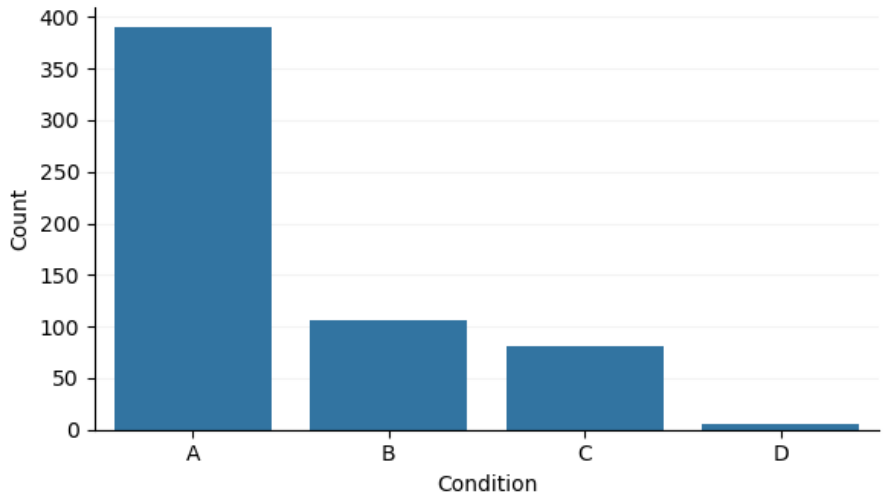

Figure 11: Distribution of bridge conditions used in the simulations.

The scenarios are summarised in Table 1. In total, nine scenarios are analysed, each representing a different probability distribution for bridge failure. In scenario 0 (base case), no bridges fail, whereas in the other scenarios the probability of bridge failure gradually increases.

Table 1: Probability distributions used for bridge failure across scenarios.

Scenario	Cat A (%)	Cat B (%)	Cat C (%)	Cat D (%)
0	0	0	0	0
1	0	0	0	5
2	0	0	0	10
3	0	0	5	10
4	0	0	10	20
5	0	5	10	20
6	0	10	20	40
7	5	10	20	40
8	10	20	40	80

For each scenario, ten simulation runs were performed, using different seeds (12–21). Using specified seeds ensures that the experiment is reproducible and the output stays consistent for individual runs. The average driving time across the ten runs is calculated and used as the model output. Ten simulation runs may be considered relatively limited, this is discussed further in Section 5.1.

Depending on the condition of the bridge, a specific probability distribution is assigned to represent the likelihood of failure. The probability distributions and corresponding delay times used in the model are presented in Table 2.

Table 2: Repair time distributions based on bridge length.

Bridge length	Delay time for a truck
Under 10 m	Uniform(10, 20) minutes
Between 10 and 50 m	Uniform(15, 60) minutes
Between 50 and 200 m	Uniform(45, 90) minutes
Over 200 m	Triangular(1, 2, 4) hours

4 Results & Discussion

4.1 Results of scenarios

The boxplot in Figure 12 shows the distribution of the performance of the scenarios and their associated driving times.

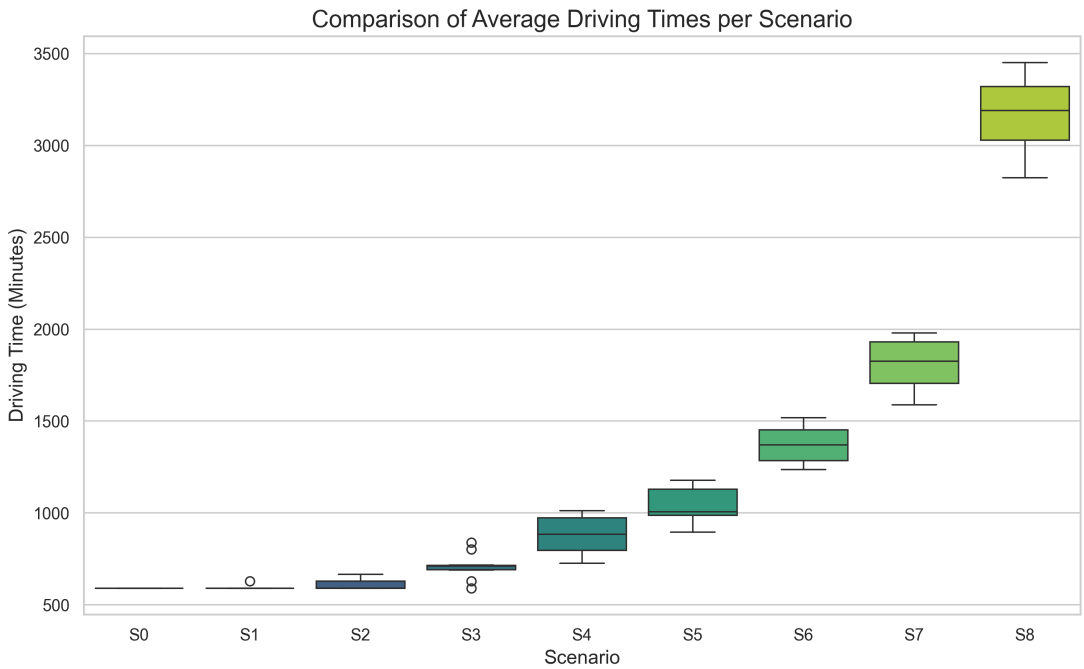

Figure 12: Distribution of driving times across eight scenarios and the basecase on the N1 highway.

In the base case, scenario 0, the average driving time remains stable at approximately 591 minutes (approximately 10 hours). With verifying the average driving time from the beginning of the N1 highway (Jatrabari, Dhaka) until the end (Teknaf in Cox's Bazar), we found that 10 hours is found as a reasonable average driving time when considering the entire road. (Verified with course staff)

The results show a clear increase in average driving time as the bridge failure probabilities increase across the scenarios. In the most reliable scenarios (S1-S3), the average driving time remains relatively close to the base case travel time. The boxplots of these scenarios are narrow, indicating low variability between the different simulation replications.

From scenario 4 onwards, the average driving time begins to increase more significantly. This reflects the growing probability of bridge failures, particularly for Category C and Category D bridges. As more bridges experience breakdowns, trucks are more likely to encounter delays, leading to longer travel times.

In the later scenarios (S6-S8), the average driving time increases substantially. Scenario 8 shows the highest travel times, with the median value exceeding 3000 minutes. This high increase is caused by the high probability of failure across all bridge categories, including the more critical Category A and B bridges. When these longer bridges fail, trucks experience much longer delays associated with longer bridges .

4.2 Differences and comparison between scenario outcomes

A key difference between the scenarios is the rate at which travel time increases as bridge reliability decreases. In the early scenarios, only lower-quality bridges (category C and D) have a significant probability of failure. Since these bridges generally cause shorter delays, the overall impact on travel time remains limited.

However, once failure begin to occur in higher-quality bridges (category A and B), the impact becomes much more significant. These bridges are often longer and therefore produce longer delays when they break down. As a result, scenarios with higher failure probabilities for these bridge categories show a much stronger increase in average travel time.

Another important difference between scenarios is the variation between simulation replications. In scenarios with low failure probabilities, the simulation results are highly consistent, resulting in small boxplots and short whiskers. In contrast, high-failure scenarios show much larger spreads in driving times. This occurs because the breakdown of a few critical bridges can significantly affect the travel time of trucks, leading to large differences between simulation runs.

Overall, the results demonstrate that the reliability of bridges plays a crucial role in determining traffic performance on the road network of N1 in Bangladesh. Even moderate increases in bridge failure probabilities can lead to substantial increases in travel time, especially when critical bridges are affected.

5 Reflection

5.1 Reliability of replications

For this assignment, ten replications were performed for each scenario. In scenarios with low bridge failure probabilities, such as scenario 1 and 2, the results are stable and the boxplots show small spreads, indicating that 10 replications are likely sufficient to estimate the mean travel time reliably. In contrast, scenarios with higher bridge failure probabilities, such as scenario 7 and 8, show much larger variability, with wider interquartile ranges and longer whiskers. This indicates that the outcomes strongly depend on which bridges fail during a simulation run, meaning that 10 replications may not be sufficient and more replications would improve the reliability of the results. As noted in literature, replicating simulation results is a key technique to increase the accuracy and validity of the model. (Dong 2019)

5.2 Limitations and possible improvements of the model

One limitation of the current implementation is how bridge lengths are represented. Distances between infrastructure components are calculated using chainage values from the dataset, which can cause bridge lengths to be counted twice: once within the road segment and once as a bridge component. This slightly overestimates total travel distance and travel time. A possible improvement is to subtract the bridge length from the corresponding road segment so that each part of the network is represented only once.

Another limitation is the use of a minute-based time-step simulation. In this approach, all agents are activated at every step, even when no relevant events occur, which reduces computational efficiency—especially with long travel times, many agents, or extended waiting periods. An improvement would be to switch to an event-based simulation using Mesa's event scheduler. Agents would only be activated when relevant events occur for them specifically, such as when a truck reaches a bridge and a new activation event is only scheduled after its waiting time. This would improve efficiency and scalability.

Finally, the model only considers delays caused by bridge failures and excludes other factors affecting travel time, such as traffic congestion, vehicle interactions, and rerouting behavior. Future work could extend the model to incorporate these additional real-world dynamics.

References

Dong, John Qi (2019). "Numerical data quality in simulation research: A reflection and epistemic implications". In: *Decision Support Systems* 126. DOI: 10.1016/j.dss.2019.113134.

Hofmann, Marko A. (2004). "Criteria for Decomposing Systems Into Components in Modeling and Simulation: Lessons Learned with Military Simulations". In: *SIMULATION* 80.7-8. DOI: 10.1177/0037549704049876.

Marsden, James R. and David E. Pingry (2018). "Numerical data quality in IS research and the implications for replication". In: *Decision Support Systems* 115, A1–A7. ISSN: 0167-9236. DOI: 10.1016/j.dss.2018.10.007. URL: https://www.sciencedirect.com/science/article/pii/S0167923618301647.

Acknowledgements

AI Statement

This report was developed with the support of AI tools. However, all ideas, prompts, and inputs originated from the authors. AI was used only as a supporting tool in two ways:

- Coding: AI assisted in resolving coding errors when the team encountered problems they could not solve independently. Prompts were kept specific, as the team understood the data and intended analyses but occasionally needed support with implementation.
- Writing: AI was also used during the writing phase to improve the flow, clarity, and structure of the text. The content, analysis, and conclusions remain the work of the authors.

Group Statement

Underneath, the role and contribution of each group member is described. Overall, the assignment was completed as a group effort, and everyone contributed to the final version of the report.

- Niels van den Dool: Set up the Git repository and LaTeX report, and mainly worked on adjustments to the model code and running simulations, wrote section 2.
- Eli Kapteijn: Primarily worked on the Mesa code, contributed to data cleaning and preparation and was involved in checking the whole report.
- Nick Schreurs: Analysed the Mesa code and suggested improvements, and mainly worked on the report, especially the scenario analysis.
- Lieke Wahlen: Focused on data exploration and preparation of the road and bridge data, and wrote the data preparation section of the report.
- Mirte Wildeboer: Contributed to early bridge data exploration and mainly wrote the results and limitations sections of the report.